



智能生成系统开发

--前言

杨卫兵



第一部分

人工智能发展历史

IT行业发展

- 个人电脑阶段：从1980年开始，以PC终端、桌面操作系统为特征产品
- 互联网阶段：以2G移动网络和宽带网络、PC终端为主要载体，软件产品、应用软件和门户网站为特征产品
- 移动互联阶段：以3/4G移动和高速宽带和移动终端为主要载体，云计算、社交网络为特征产品
- 人工智能阶段：以AlphaGo（2016年）为标志
- 未来：

人工智能发展

- 1950年，图灵测试
- 1996年，IBM，深蓝（Deep Blue），卡斯帕罗夫
- 2014年，尤金·古斯特曼（Eugene Goostman），“13岁男孩”
- 2016年，AlphaGo，李世石
- 2022年，OpenAI，ChatGPT
- 2024年7月23日，Meta发布LLAMA3.1-405B；9月9日，阿里QWen2.5-72B发布
- ⁴■ 2025年12月，DeepSeek V3.2 正式版发布

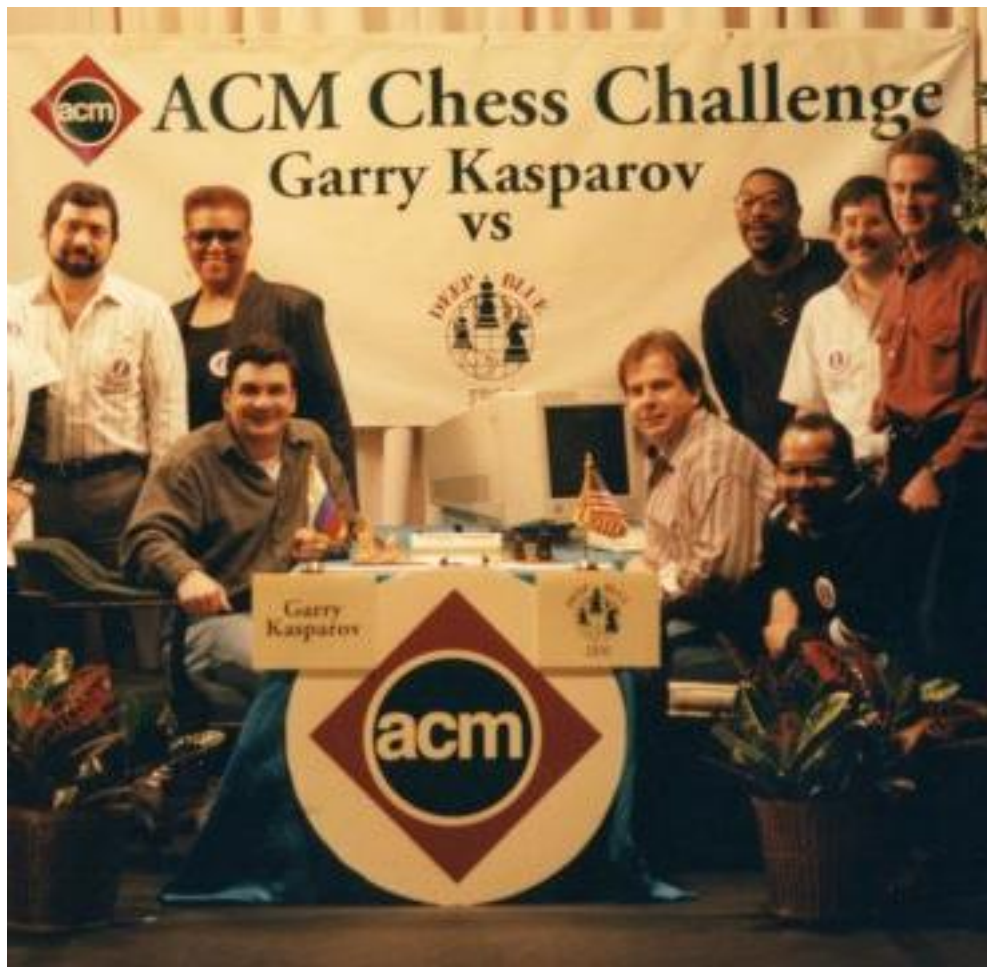
人工智能之父：艾伦·图灵



Alan Mathison Turing (1912 - 1954) , 英国数学家、逻辑学家，被称为计算机科学之父，人工智能之父。二战期间协助英国军方破解德国的著名密码系统Enigma，帮助盟军取得了二战的胜利。

图灵提出了一种用于判定机器是否具有智能的试验方法，即图灵测试，对于人工智能的发展贡献巨大。

深蓝Deep Blue



- 1962年, IBM 7090, 每秒1100步
- 1996年2月, 2-4
- 1997年5月, 3.5-2.5, 32个处理器, 每秒2亿步计算能力, 11.38GFLOPS

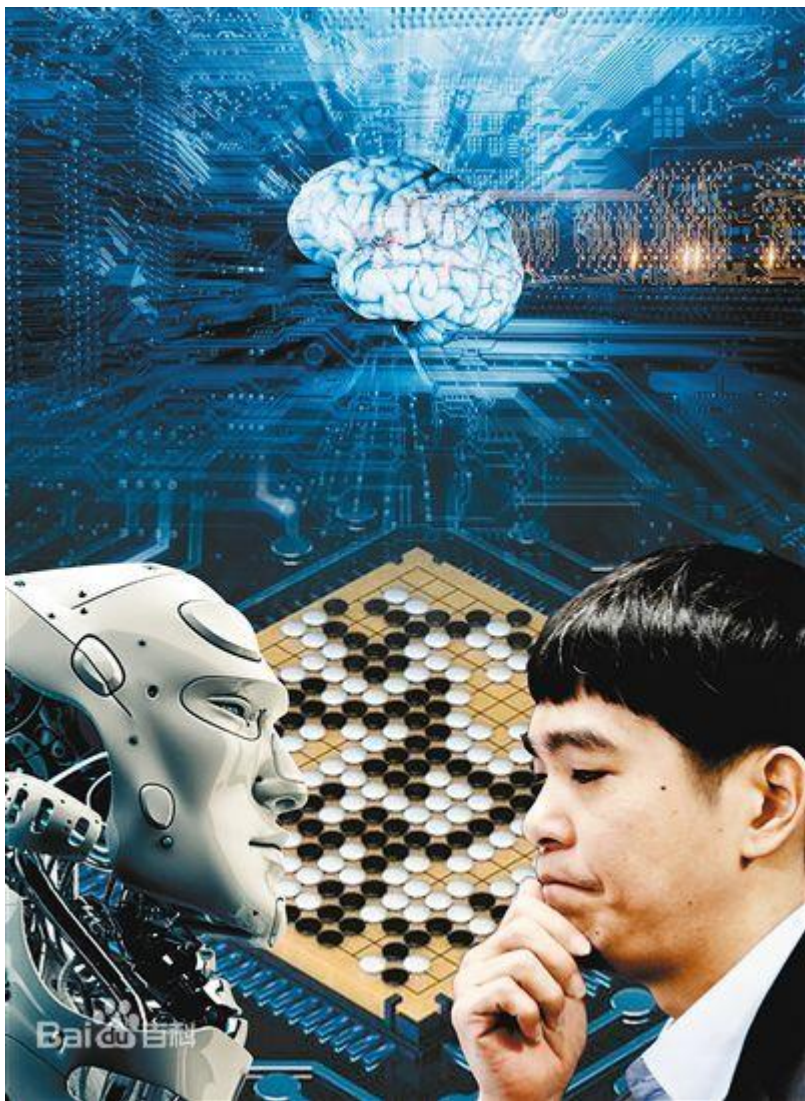
尤金·古斯特曼



- 2001年，诞生于俄罗斯圣彼得堡
- 2014年6月7日，图灵逝世60周年纪念日，通过“图灵测试”

Baidu 百度

AlphaGo



- DeepMind公司, 深度学习
- 2016年10月, 5: 0欧洲冠军
- 2016年3月, 4: 1李世石
- 2016-2017: Master, 60局无一失败
- 2017年5月, 3: 0柯洁
- 2017年10月, AlphaGo Zero

OpenAI和ChatGPT



- 2015年，由萨姆·奥尔特曼、彼得·泰尔、里德·霍夫曼和埃隆·马斯克等创办
- 2018年，马斯克退出
- 2019年，转为“封顶”营利性公司；微软注资10亿美元
- 2022年11月，ChatGPT问世
- 2024年5月，GPT-4o，多模态

人工智能相关技术

- 算法
- 大数据
- 算力
- 大模型
- 应用开发



算法

- 回归、分类、聚类
- 决策树：ID3、CART
- Bagging、随机森林
- 线性回归、过拟合、岭回归、逻辑回归
- SVM、朴素贝叶斯、K-means
- 神经网络、CNN
- 深度学习、.....

大数据

- Mnist数据集

60000+10000

- ImageNet

ILSVRC: 1400万图片

- AlphaGo

数百万人类围棋专家的棋谱



谷歌数据中心



算力

■ 通用算力计算公式

$$E_t = 8 * T * P / (n * X)$$

参数量P (B)	训练阶段	数据量T (B tokens)	卡数 n	训练时长 (天)
175 (e.g. ChatGPT)	预训练	3500 ^③	8192	49
	二次训练 ^②	100	2048	5.5
110 (e.g. GPT-3)	预训练	2000	4096	35
	二次训练	100	1024	7
65 (e.g. LLaMA)	预训练	1300	2048	27
	二次训练	100	512	8
13 (e.g. LLaMA)	预训练	1000	256	34
	二次训练	100	128	7

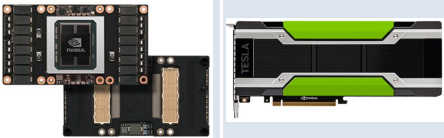
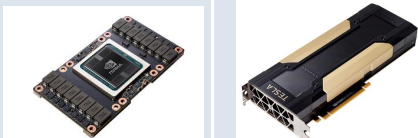
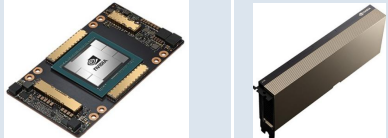
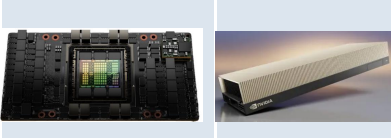
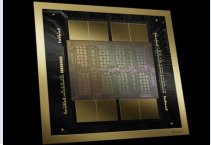
算力集群规模的整体要求是：

百亿参数用百卡

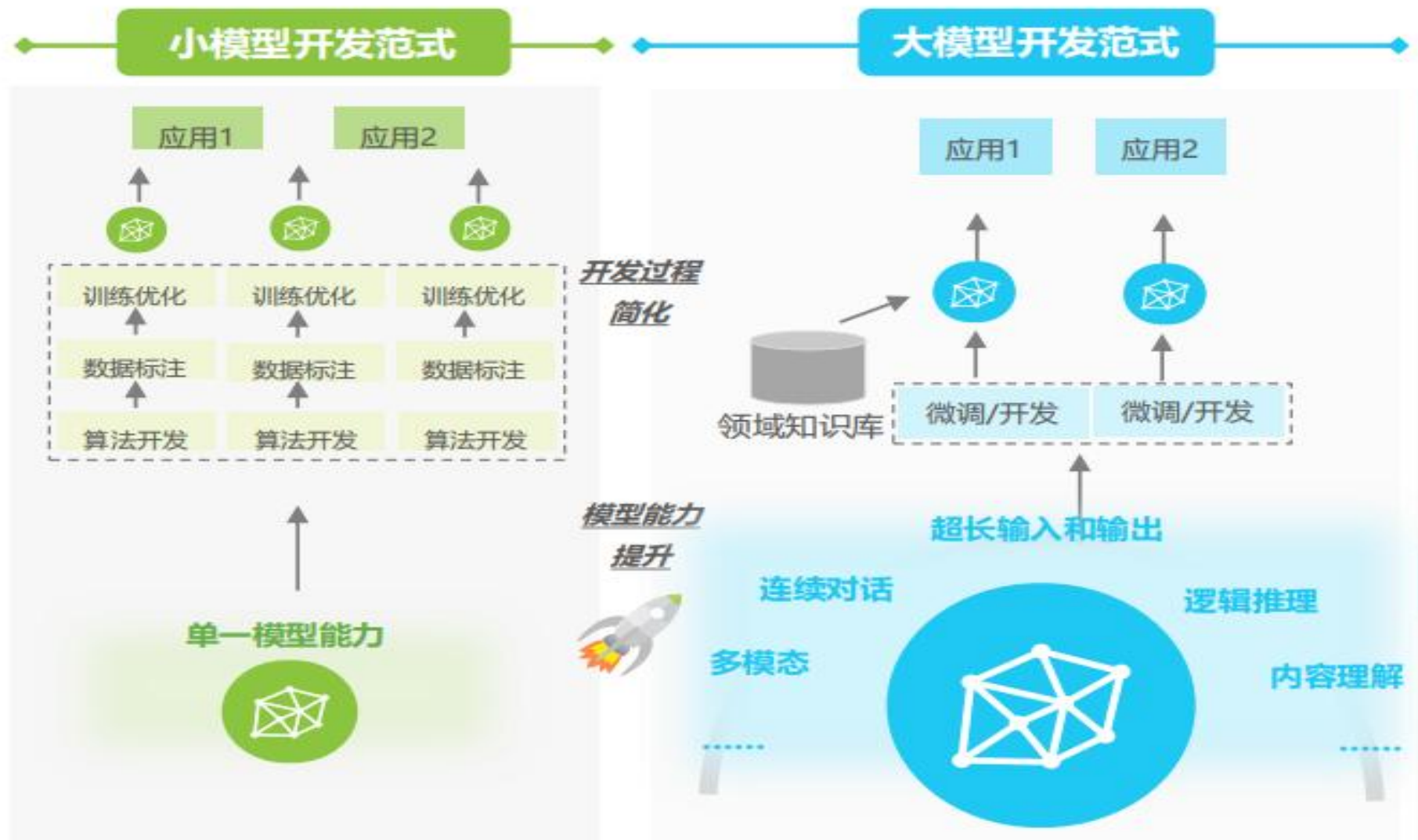
千亿参数用千卡

万亿参数用万卡

英伟达GPU

类别名称	P100		V100 (S)		A100		H100		B200
产品图片									
发布时间	2016年Q2		2017年Q2、2019年Q4 (S)		2020年Q2		2022年Q1		2024年Q1
架构代号	Pascal		Volta		Ampere		Hopper		Blackwell
算力性能 TFlops	FP64-5.3 FP32-10.6 FP16-21.2	FP64-4.7 FP32-9.3 FP16-18.7	FP64-7.8 FP32-15.7 FP16-125	FP64-7/8.2 FP32-14/16.4 FP16-112/130	FP64-9.7/19.5(Tensor) FP32-19.5/156(Ten)/312* FP16-312/624* FP8-624/1248* A100的PCIe和SXM单卡性能一致		FP64-34&26 (标准) FP64-67&51 (Tensor) FP32-67&51 (标准) FP32-989&756 (Tensor) FP16-989&756 FP16-1979&1513 (稀疏) Int8是FP16的两倍		FP64-40 (标准) FP32-1120&2250 (稀疏) FP16-2250&4500 (稀疏) Int8-4500&9000 (稀疏) FP8-4500&9000 (稀疏) FP4-9000&18000 (稀疏)
工艺制程	16nm		12nm		7nm		4nm		4nm
显存/带宽	16 GB HBM2 732GB/s	16GB HBM2 732 GB/s	32/16 GB HBM2 732GB/s, 1134GGB/s (S)		80GB HBM2e 2039GB/s, 1935GB/s		80GB HBM3&HBM2e 3.35TB/s、2TB/s		192GB HBM3e 8TB/s
NvLink	NL 1.0 x4 160GB/s	-	NL 2.0 x6 300GB/s	-	NL 3.0x12 600GB/s	-	NL 4.0x18 900GB/s	-	NL 5.0x18 1800GB/s
PCIe	PCIe 3.0x16、32GB/s		PCIe 3.0x16、32GB/s		PCIe 4.0x16、64GB/s		PCIe 5.0x16、128GB/s		PCIe 5.0x16、128GB/s
TDP	300W	250W	300W	250W	400W	300W	700W	350W	1000W
备注	NvLink 1.0	-	Tensor Core		MIG、Nvlink 桥接		NVLINK 4.0, H100NVL		PCIe版本未公布

开发模式



机器学习框架

- TensorFlow: 2015年11月, Google正式开源
 - tensor: 张量, 一个多维数组, 基础数据模型
 - flow: 流, 张量之间通过计算相互转换的过程
- PyTorch: 2017年1月, Facebook (现Meta) 推出
 - 动态计算图
 - 在科研领域, 占有率80%

主流大模型（国外）

- OpenAI: 2020年GPT-3 (175B) , 训练数据45TB;
GPT-4未公布 (估计是100倍)
- Meta: LLAMA 3.1-405B(128K), 2024年7月, 16384
张H100训练N个月, 耗资数亿美元, 230G
- xAI: Grok 1-314B, 2024年3月
- Anthropic: Claude 3.5 Sonnet, 2024年6月; 同年8
月推出 “提示缓存” 功能, 大幅降低使用成本

主流大模型（国内）

- 百度：文心一言(ERNIE Bot)，2024年6月发布4.0 Turbo
- 华为：盘古，2024年6月发布5.0，最多万亿级参数
- 阿里：通义千问(QWEN)，2024年9月发布2.5-72B
- 腾讯：混元，2024年9月发布Turbo版
- 智谱：ChatGLM，2024年9月发布GLM-4-plus
- 科大讯飞：星火，2024年6月发布V4.0



其它AI大模型

- 语音：Deepgram的Aura、Amazon的Polly等
- 绘画：Midjourney、Stable Diffusion 等
- 音乐：Suno、天工SkyMusic等
- 视频：Sora、可灵等

高考全卷评测

- 2024年6月，司南评测(OpenCompass)选取6个开源模型及OpenAI的GPT-4o开展首个大模型高考（语数外）能力测试
- Qwen2-72B、GPT-4o及书生·浦语2.0文曲星（InternLM2-20B-WQX）成为三甲，对应得分率分别为72.1%、70.5%和70.4%
- 数学是所有大模型的短板，平均得分率仅为36%；最高的文曲星也仅得75分，未达到及格水平。



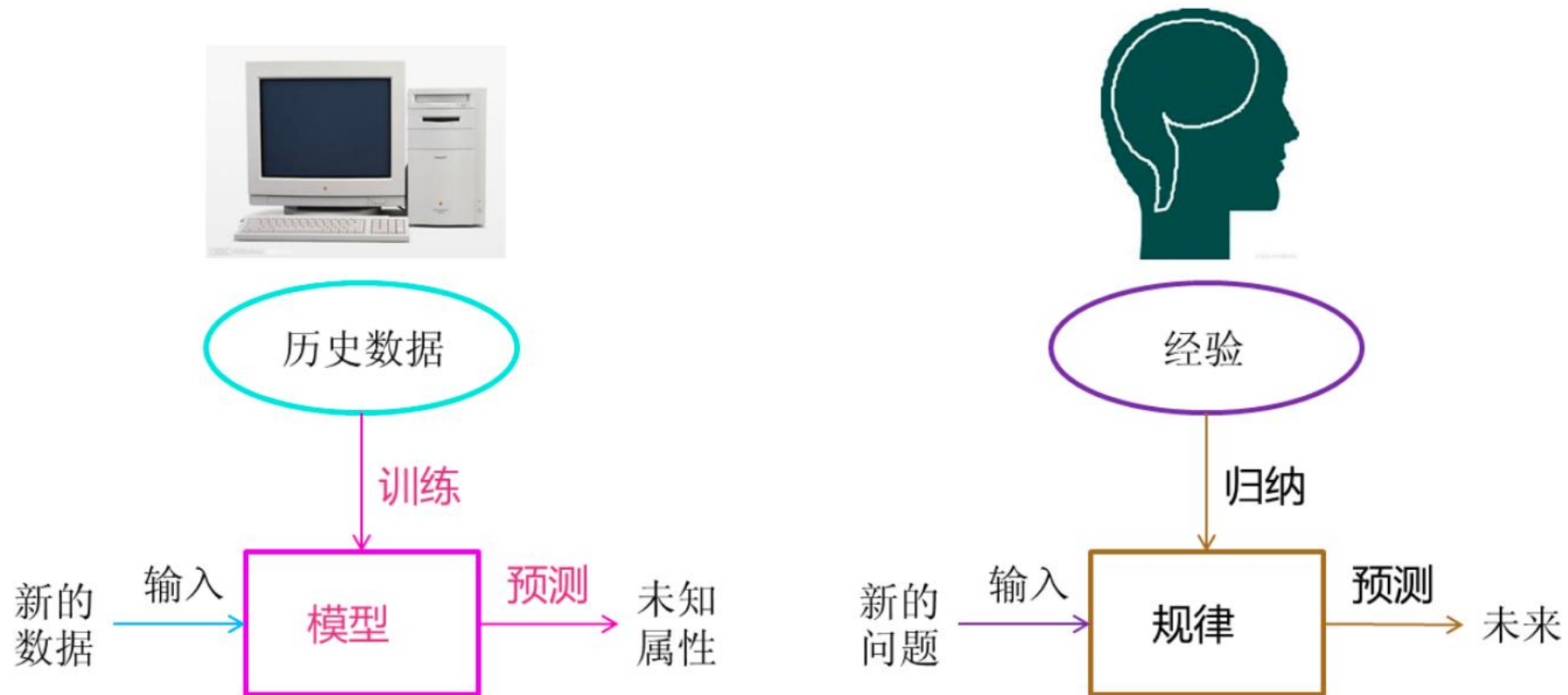
第二部分

机器学习

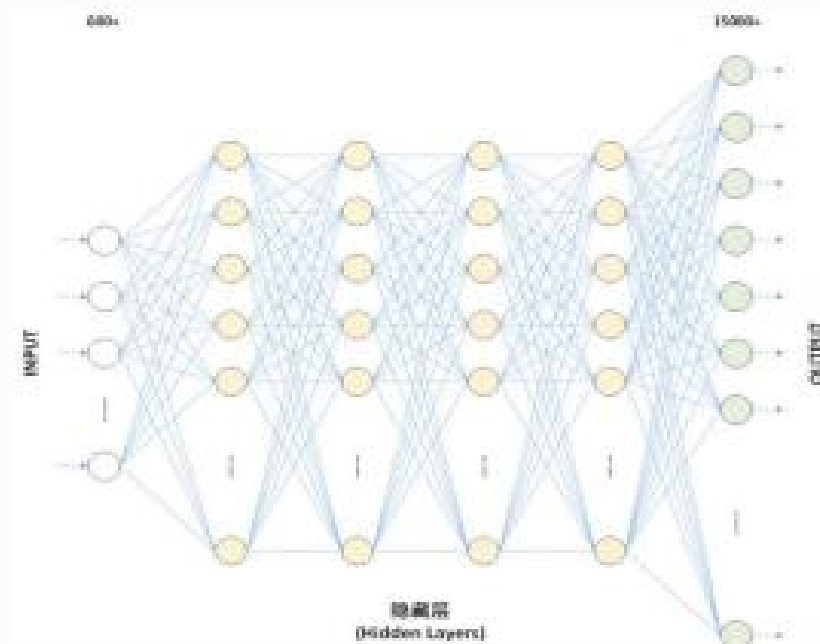
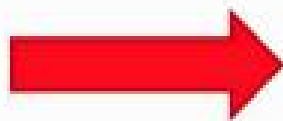
人工智能/机器学习/深度学习



机器学习与人类思维

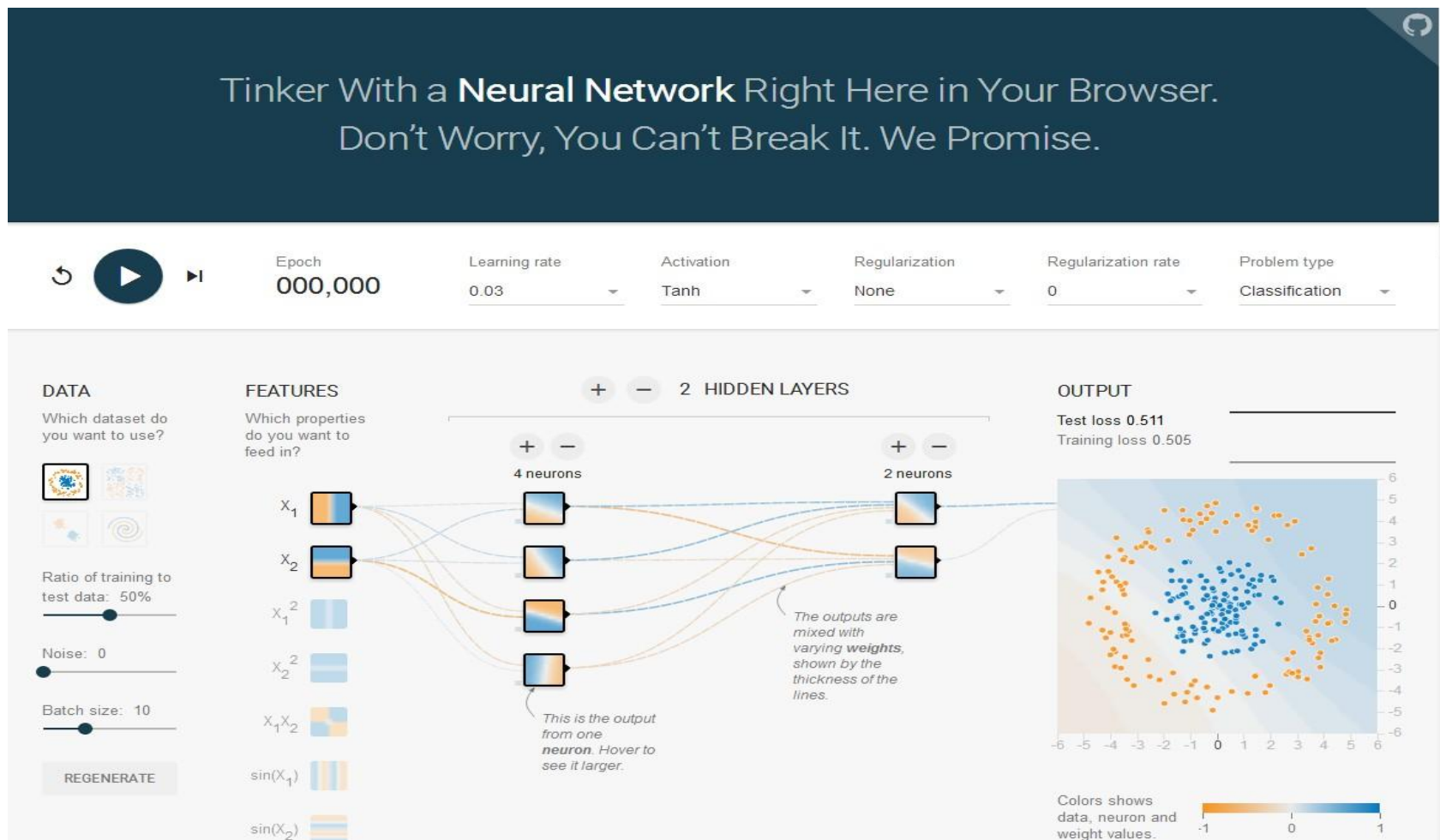


神经网络 (NN)

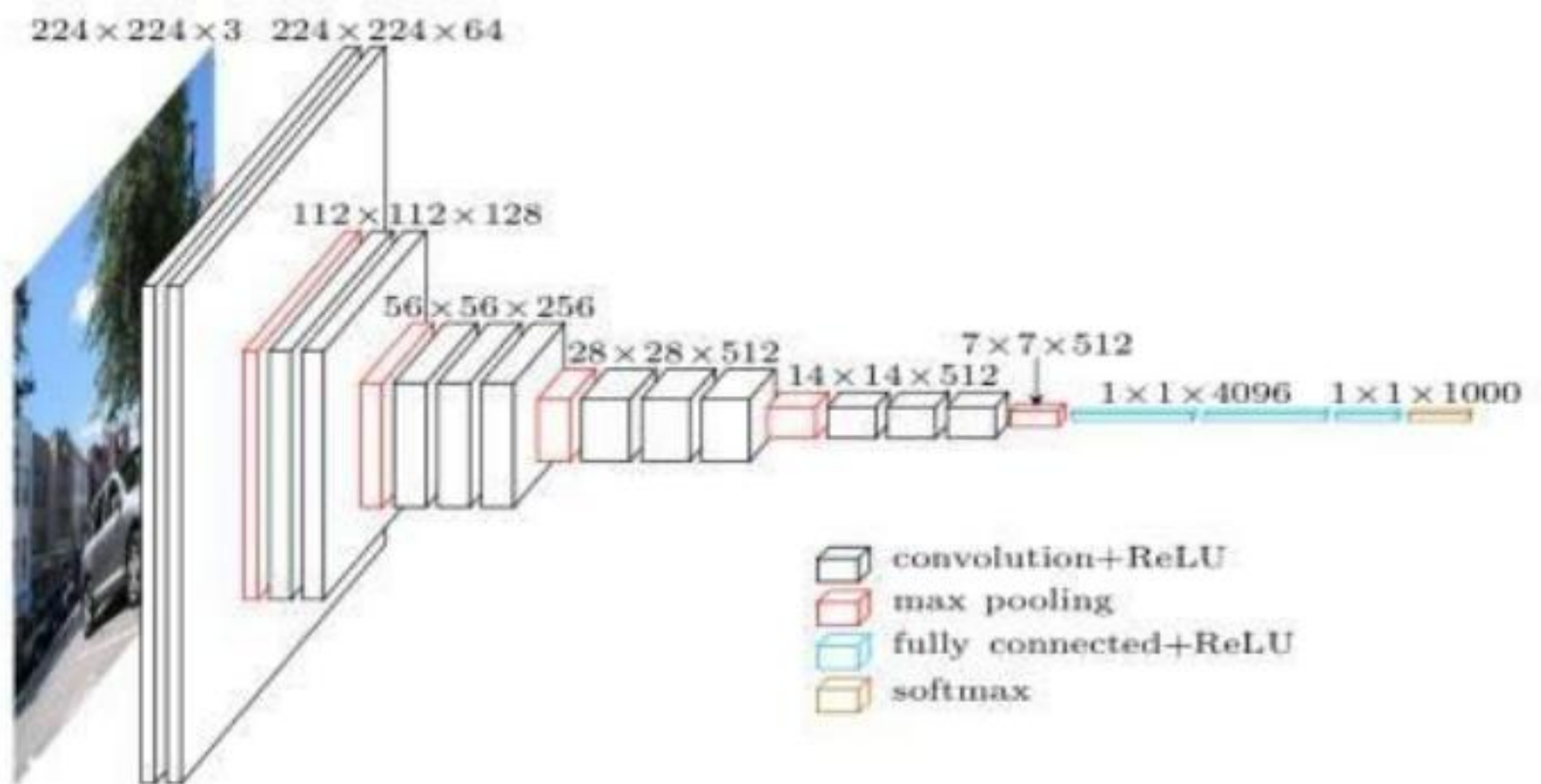


Google神经网络演示平台

<http://playground.tensorflow.org/>



深度学习 (DL)



Geoffrey Hinton

- 英国出生的计算机学家和心理学家，以其在神经网络方面的贡献闻名。Hinton是反向传播算法的发明人之一，也是深度学习的积极推动者。目前担任多伦多大学计算机科学系教授。
- 2013年3月加入Google，领导Google Brain项目。



Yann LeCun

- 计算机科学家，他最著名的工作是光学字符识别和计算机视觉上使用卷积神经网络（CNN），他也被称为卷积神经网络之父。
- 多伦多大学跟随Hinton做博士后。1988年，加入贝尔实验室，之后研发出了卷积神经网络，曾广泛用于手写数字识别。
- 2003年去了纽约大学任教。2013年12月加入了Facebook，成为Facebook人工智能实验室的第一任主任。



Yoshua Bengio

- 在MIT和贝尔实验室做过博士后研究员，自1993年之后就在蒙特利尔大学任教。在预训练和自动编码器等方面作出过重大贡献。
- “我留在学术圈为全人类作贡献，而不是为某一个公司赚钱”

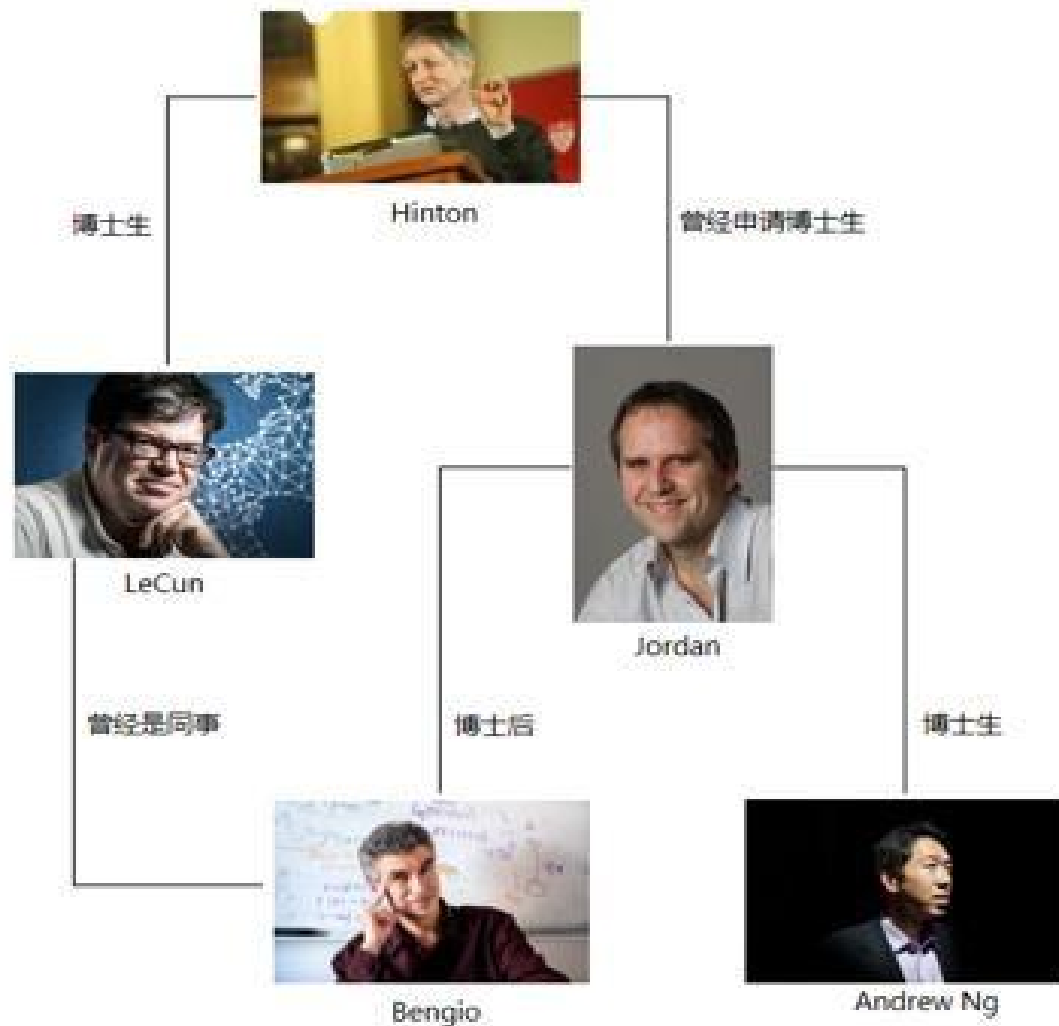


Andrew Wu (吴恩达)

- 曾经是斯坦福大学计算机科学系和电气工程系的副教授，斯坦福人工智能实验室主任。他创建了在线教育平台 Coursera。
- 2011年，吴恩达在Google创建了Google Brain项目，通过分布式集群计算机开发超大规模的人工神经网络。
- 2014年5月，吴恩达加入百度，负责百度大脑计划，并担任百度公司首席科学家。
- 2017年3月，吴恩达从百度离职。



关系图



LeCun Hinton Bengio Andrew Ng

知识框架



机器学习过程

- 建模之前，我们可以把数据分成三部分。

训练集(Training data)

验证集(Validation data)

测试集(Test data)

- 训练集还是用来训练，构建模型。
- 验证集是用来在模型训练阶段测试模型的好坏。
- 等模型训练好之后，再用测试集来评估模型的好坏。

CNN示例：MNIST

- MNIST (Mixed National Institute of Standards and Technology database) 数据集是机器学习最有名的数据集之一，尤其在深度学习领域
- Mnist数据集包含训练集（包含60,000张图像和对应的标签，用于模型训练）和测试集（包含10,000张图像和对应的标签，用于模型评估）两部分
- 所有图像都是28x28的灰度图像，像素值范围为0-255。标签为0-9之间的整数，代表图像中的手写数字
- 在实际应用中，使用MNIST数据集进行手写数字识别通常包括以下步骤：数据加载、数据预处理（包括归一化、重塑数据形状等）、模型构建（如CNN）、模型训练（使用训练集）、模型评估（使用测试集）、结果分析（根据评估结果对模型进行调优和改进）



第三部分

大语言模型 (LLM)

基本概念

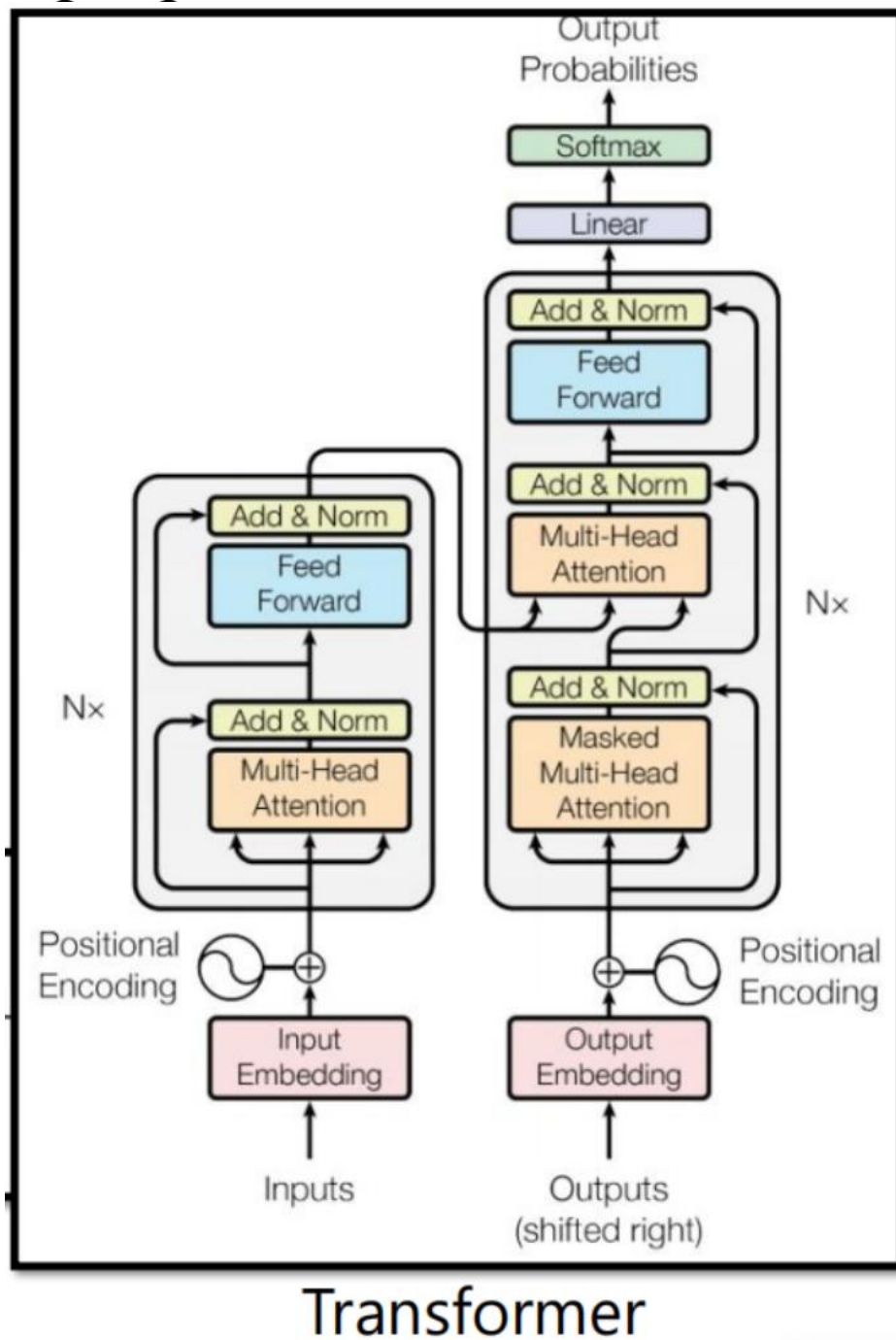
- 大语言模型（Large Language Model，简称LLM），指使用大量文本数据训练的深度学习模型，可以生成自然语言文本或理解语言文本的含义。
- 大语言模型可以处理多种自然语言任务，如文本分类、问答、对话等，是通向人工智能的重要途径。
- 目前大语言模型采用与小模型类似的Transformer架构和预训练目标（如 Language Modeling），与小模型的区别是增加模型大小、训练数据和计算资源。

重要事件

- 2013年，自然语言处理模型 Word2Vec诞生，首次提出将单词转换为向量的“词向量模型”，以便计算机更好理解和处理文本数据。
- 2014年，GAN(对抗式生成网络)诞生，被誉为21世纪最强大算法模型之一，标志深度学习进入生成模型研究的新阶段。
- 2018年，Google AI研究院提出了BERT(预训练+参数微调范式)；同年，OpenAI公司同样发布了自己的模型GPT (Generative Pre-Training)，一个典型的生成式预训练模型。
- 2020年，OpenAI发布GPT-3，意味着NLP领域的大语言模型真正意义上出现了，从此正式开启大语言模型时代。

Transformer

- Transformer是2017年谷歌的研究人员在论文“Attention Is All You Need”中首次提出，并将其应用于机器翻译任务。
- Transformer是一种用于序列到序列任务的神经网络模型，如机器翻译、语音识别和生成对话等。
- 序列到序列模型采用的是编码器-解码器结构



大模型训练过程

■ 有监督学习 (Supervised Learning)

- generate: 先使用大量语料进行无监督学习, 训练出一个语言模型的基座
- chat: 人工整理QA语料对大模型进行有监督训练

■ 奖励模型训练 (Reward Model)

- 根据一个问题不同答案的优劣进行评分, 再通过回归模型进行反向传播

■ 强化学习 (Reinforcement Learning)

- 模型可以自我迭代

通用大模型应用

- Web端/应用程序(APP)
- 通过大模型厂商提供的API
 - SDK
 - Http/Https

专业大模型应用

- 行业解决方案
 - 度小满的轩辕系列
 - 快手的可灵

大模型微调

- **大模型微调**是指在已有大模型基础上，使用新的、与特定任务相关的数据集对模型进行进一步训练的过程
- 目的：使模型能够**快速适应新的任务或领域**，同时保持大模型强大的特征提取能力
- 分类：
 - 全面微调
 - 参数高效微调（PEFT）：通过最小化微调参数数量和计算复杂度，提升**预训练模型**在新任务上的表现，从而减轻大型预训练模型的训练负担。

微调步骤

- 配置Python环境: Python 3.10+
- 选择一个预训练模型下载并安装所需软件包
- 数据准备和加载
- 定义任务和损失函数
- 微调模型
- 评估模型
- 调整参数
- 保存和部署

Llama-Chinese / requirements.txt 

 zz update

Code

Blame

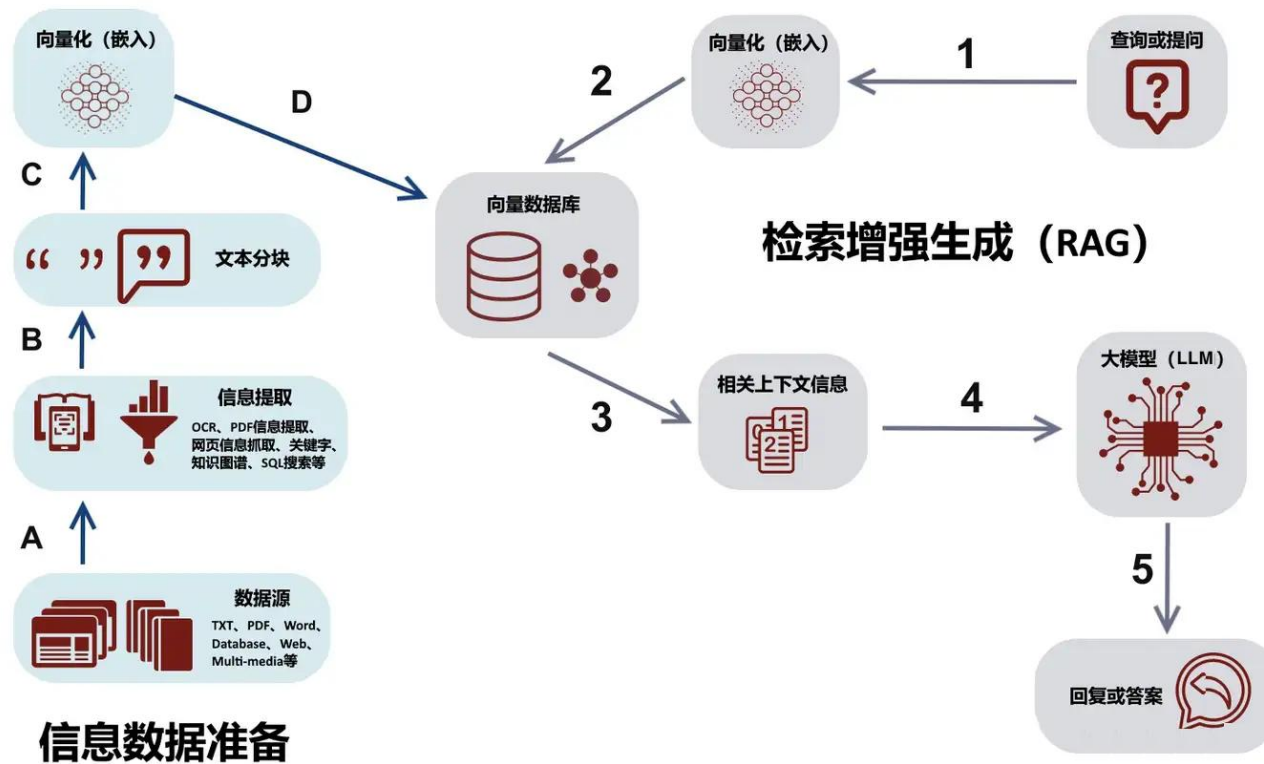
21 lines (21 loc) · 264 Bytes

```
1 torch==2.1.2
2 bitsandbytes==0.42.0
3 accelerate==0.27.2
4 numpy==1.26.4
5 gekko==1.0.6
6 pandas
7 scipy
8 sentencepiece==0.2.0
9 datasets
10 evaluate
11 pytest
12 peft==0.8.2
13 transformers==4.39.0
14 deepspeed==0.14.0
15 scikit-learn
16 torchvision
17 torchdata
18 torchaudio
19 tensorboard
20 gradio
21 packaging
```

RAG

■ 检索增强生成 (RAG: Retrieval-Augmented Generation) 是一种结合了检索和生成的方法

- 增强生成能力
- 提高事实准确性
- 解决长程依赖问题
- 降低模型复杂度
- 适应特定领域
- 提高效率



微调 vs RAG

RAG

知识更新	实时、低成本
推理速度	较慢（需检索）
成本	低（无需训练）
风格定制	弱（依赖提示工程）
可解释性	强（可追溯源文档）
适用场景	知识频繁更新、需可追溯的问答

微调

困难、高成本
快（直接生成）
高（训练昂贵）
强（深度内化）
弱（黑箱）
风格固定、低延迟的特定任务

RAG与微调的混合架构

■ 核心思想：

- 让微调负责“怎么说”：教会AI你公司的语言风格、报告模板、行文规范。
- 让RAG负责“说什么”：为AI提供实时、准确、具体的业务知识和事实数据。

■ 混合架构工作流：

用户提问：“为‘A流程’生成测试用例。”

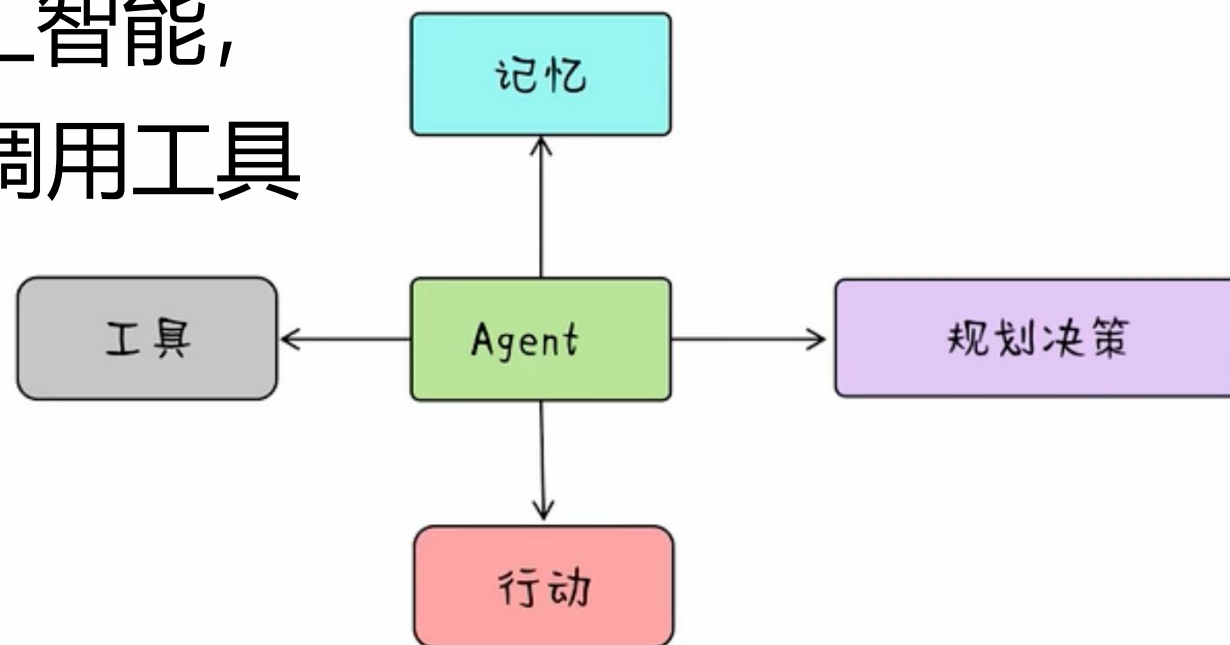
RAG侧工作：从向量数据库中检索出关于“A流程”的最新需求文档、API变更说明。

微调侧工作：将“检索到的资料”和“用户问题”组合成一个结构化的Prompt，输入给经过微调的专属模型。

最终生成：微调模型基于Prompt生成回答。因为它既接收了最新的业务信息（来自RAG），又内化了标准的输出格式（来自微调），所以能产生格式规范、内容准确的测试用例。

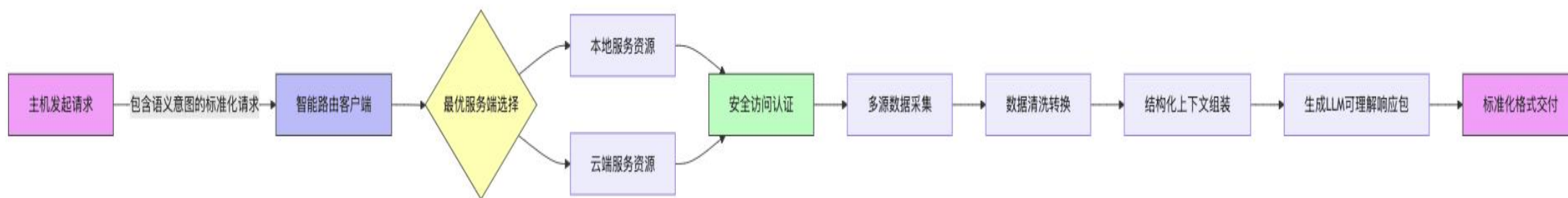
Agent

- 能够感知环境、进行决策和执行动作的智能实体。它可以是软件、硬件或一个系统，具备自主性、适应性和交互能力。
- Agent不同于传统的人工智能，它能够通过独立思考、调用工具去逐步完成给定目标。



MCP

■ 模型上下文协议（MCP: Model Context Protocol）是由Anthropic于2024年11月推出的**开源协议**，旨在实现大语言模型与外部数据源和工具的集成，**用来在大模型和数据源之间建立安全双向的连接**。该协议通过**相同的协议同时处理本地资源**（例如数据库、文件、服务等）和**远程资源**（例如Slack或GitHub等API）



Skills

- Skills 是2025年10月，Anthropic在Claude Code上支持的新特性，是Agent能够执行的具体、原子化的能力或工具。
 - 可复用的功能模块：一段封装好的代码或 API 调用，专门做一件事。
 - 大模型的“手和脚”：Agent 通过大模型的决策能力，来决定什么时候调用哪一个 Skill。
 - 与 Tools（工具）同义：在很多框架里，Skills 和 Tools 这两个词经常混用，都指代模型可以调用的外部功能。

关系

- Agent: 大脑+调度中心, 负责思考: 用户想干什么? 第一步做什么? 第二步做什么? 遇到问题怎么办?
- Skills: 肌肉+具体动作, 被大脑调用的具体能力。比如大脑说“需要查天气”, 肌肉就去执行“调用天气API”这个动作。
- RAG: 知识库检索员, 是一种特殊的、非常核心的 Skill。这个 Skill 专门负责去向量数据库里查找相关资料, 然后拿回来给 Agent 看。
- MCP: 神经接口+标准化插头, 让 Agent 能够更方便地拥有 Skills 的方式。通过 MCP 协议, Agent 可以即插即用地获取各种 Skill, 而不需要为每个 Skill 单独写代码。



第四部分

RAG

引言

- ◆ 检索增强生成（RAG）技术正迅速成为个人与企业最常用的人工智能应用方案，该技术将数据检索能力与生成式大型语言模型（LLM）相结合。
- ◆ 本课程内容：
 - 理解RAG的基本概念及其在人工智能领域的应用
 - 掌握嵌入技术和向量数据库在RAG系统中的核心作用
 - 明晰RAG与传统大语言模型方法的本质区别

技术背景

- ◆ RAG是一种通过集成动态数据检索机制显著增强LLM能力的技术。作为人工智能领域的重要应用，它充分利用LLM的自然语言处理能力，为各类实际应用生成精准的上下文相关信息。
- ◆ 当前新兴技术中涌现的众多AI应用，本质上都是经过微调优化的RAG变体。这类技术能够轻松整合多源定制数据，并与GPT（生成式预训练Transformer）等成熟语言模型（如ChatGPT所采用的模型）协同工作——这种低门槛特性使其成为开发者构建强大AI应用的理想选择。

LLM的其它应用场景

- ◆ Agent：能够代表用户执行指令的应用程序，典型代表有AutoGPT和BabyAGI。
- ◆ 提示工程（Prompt Engineering）：通过精心设计LLM输入提示词来获得更优质的输出响应。LLM本质上受限于其训练数据与方法论，用户提问或指令的表述方式直接影响着模型输出质量。
- ◆ 零样本/单样本/少样本学习：LLM的另一种创新应用形式。开发者可通过提供无示例、单一示例或多个示例的提示词，引导模型完成特定任务。
- ◆ 模型蒸馏：该技术致力于创建更精简高效的模型版本。通过聚焦特定场景（如图书馆本地数据或区域气象数据），使得模型能够在内存、存储空间和算力受限的小型设备上流畅运行。

RAG的重要性-1



LLM通常基于公开数据集进行训练，因此当它们表示无法获取你的个人财务记录或社交网络活动信息时，这种局限性是完全可以理解的。

01



这种局限性意味着聊天应用主要适用于通用知识领域，这难免令人遗憾。倘若这些AI聊天应用能突破现有知识边界，实现更多可能性该有多好。这正是RAG技术的价值所在。

02



RAG通过开发者输入的数据增强了LLM的能力边界。这意味着采用RAG技术后，你既能享受与ChatGPT等大语言模型流畅对话的体验，又能使其具备特定场景的认知能力。

03

RAG的重要性-2



RAG技术开启了无限可能：它能提升LLM在任意场景下的准确性、可靠性、灵活性、知识储备、运行效率、动态适应性和场景相关性，强化信息检索能力，从根本上降低AI应用的错误概率。

04



当LLM对特定主题缺乏认知时，可能呈现三种应答模式：坦诚表达无能为力并致歉、信誓旦旦提供虚假信息、或是给出完全答非所问的内容。这些情况都应竭力避免，在特定场景下甚至可能引发严重后果，RAG技术能显著缓解上述所有问题。

05



RAG应用具备处理非结构化数据的能力，这意味着它可以读取来自PDF文档、网页页面、数据库及其他各类数据源的信息。如今应用生态中涌现的多数AI增强功能，本质上都在运用某种形式的RAG技术

06

RAG相关关键词

- ◆ 人工智能涵盖众多分支领域、概念体系及专业术语。本课将聚焦与RAG相关的关键词：
 - 模型（Model）：指经过训练能够识别模式并进行预测的程序系统
 - 大型语言模型（LLM）：具备人类语言文本理解与生成能力的智能模型
 - 提示词（Prompt）：以自然语言形式向LLM提出的结构化请求，旨在获取具有针对性的响应内容
 - 自然语言处理（NLP）：人工智能的重要分支领域，致力于实现计算机对人类语言的理解能力

RAG核心概念

◆ 与RAG相关的核心概念：

- 检索（Retrieval）：首先从单个或多个数据源获取非结构化文本数据，这构成了RAG中"检索"（R）部分的基础。
- 增强（Augmentation）：该概念承认现有组件（LLM）的存在，并通过利用检索过程获得的数据来增强这些LLM的能力。
- 生成（Generation）：LLM的响应本质上具有生成性。与传统软件应用基于预定义逻辑生成输出不同，LLM依托训练数据与方法，通过实时预测文本序列来创建响应。

嵌入的概念

- ◆ 嵌入（Embeddings）是一种数值化数据表征技术，旨在捕捉词语、句子乃至更大文本块的语义内涵，使其能够被LLM有效利用。
- ◆ 在RAG应用中完成数据采集后，需对数据进行嵌入处理，以便与提示词及LLM协同使用。从内部机制来看，嵌入技术会将文本转换为向量空间中的多维数值向量，这种数据格式极大简化了数据分类的流程。
- ◆ 多维数值向量使文本间的语义对比更加精确，语义相近的词汇在向量空间中的位置更为接近，从而提升模型对提示词和数据的理解能力，最终生成更精准的响应。

向量数据库

- ◆ 为有效管理嵌入技术生成的向量数据，专门研发了名为向量数据库（Vector Databases）的专业数据库，用于存储和查询向量数据。这类数据库经过深度优化，具备卓越的可扩展性和查询效率，使文本检索变得轻而易举。
- ◆ 嵌入技术与向量数据库的应用范围远不止文本数据，在图像、音频及其他数据格式的处理中同样表现优异。

嵌入分类

- ◆ 根据具体应用场景选择合适的嵌入类型至关重要。嵌入技术主要可分为两类
 - 单模态嵌入：仅接收单一数据类型作为输入，例如纯文本或纯图像。文本嵌入在文本分类、情感分析等场景中表现突出，而图像嵌入则擅长目标识别与分类任务。
 - 多模态嵌入：可处理混合数据类型，如文本与图像、表格、图表的组合。

其它嵌入分类

- ◆ 除上述两类外，还有以下几种常用嵌入类型：
 - **词嵌入**（Word Embeddings）：按语义关系存储词汇，语义相近的词汇在向量矩阵中位置相邻，而无关词汇则相距较远。
 - **句嵌入**（Sentence Embeddings）：将短语和完整句子表示为向量，而非单个词汇。典型代表是谷歌开发的通用句编码器（USE），可为各类自然语言处理任务生成固定长度的句向量。
 - **文档嵌入**（Document Embeddings）：与其他嵌入类型本质相同，但存储对象是完整文档而非仅限句子或词汇。
 - **上下文嵌入**（Contextualized Embeddings）：前三类属于静态嵌入。上下文嵌入则根据语句中的具体语境存储数据。这种机制能更精细地把握词义变化，因为同一词语在不同语境下会生成差异化的向量表征。

嵌入的重要性

- ◆ 嵌入技术使得AI系统能够高效表征各类数据。它不仅通过内存优化实现海量数据的高效存储，更为数据检索提供了创新方案。
- ◆ 嵌入技术将文本、音频、视频数据转化为数值表征的同时，还赋予了系统情感分析、文本摘要、文本生成、语音识别、音乐分类、音频生成、目标检测、视频生成等高级功能。这些突破性能力是传统数据表征方式（仅服务于基础存储与检索）所不具备的。

小结

- ◆ RAG是一种通过定制数据增强LLM能力的技术，为人工智能个性化提供了实用且高效的实现路径。该技术兼容结构化和非结构化数据，能够全面提升LLM及其他AI应用的性能表现。
- ◆ 嵌入技术将数据转化为数值化向量表征，并根据特定语境进行语义化组织。这种技术不仅实现了从文本到图像、视频等多元数据的有效存储，更赋予了系统高级检索能力。针对不同应用场景，可选用差异化的嵌入类型以实现最优数据检索效率。
- ◆ 向量数据库作为专为存储和管理嵌入向量而设计的专业数据库，其出色的可扩展性使其能够从容应对海量数据集的存储需求，在需要处理大规模信息的RAG系统中扮演着不可或缺的角色。

实践1：访问本地LLM

◆ Ollama方式

- 安装并引入依赖
- 创建会话
- 输出结果

◆ OpenAI方式

- 安装并引入依赖
- 连接本地Ollama
- 创建会话
- 输出结果

实践2：使用llama-index嵌入文件

- ◆ 安装llama-index、llama-index的Ollama依赖
- ◆ 引入相关依赖
- ◆ 设置本地模型
- ◆ 加载本地文件
- ◆ 不使用向量数据库建立索引（比较耗费系统资源）
- ◆ 创建查询引擎
- ◆ 执行查询
- ◆ 打印响应信息

实践3：在实践2中添加向量索引

- ◆ 导入相关的库
- ◆ 加载数据
- ◆ 构建索引
- ◆ 创建问答引擎
- ◆ 开始问答

实践3（改进）：自选嵌入模型和本地生成模型

- ◆ 安装Huggingface依赖
- ◆ 导入相关的库
- ◆ 加载本地嵌入模型
- ◆ 加载数据
- ◆ 构建索引
- ◆ 创建问答引擎
- ◆ 开始问答

实践4：使用Langchain

- ◆ 安装Python 3.10的虚拟环境
- ◆ 安装依赖并在代码中引入依赖
- ◆ 设置嵌入模型
- ◆ 加载网页文档并进行文档分块
- ◆ 创建向量存储（内存存储）
- ◆ 根据用户问题在向量存储中搜索相关文档，并准备上下文内容
- ◆ 构建提示模板
- ◆ 使用大语言模型生成答案
- ◆ 回答问题并输出

实践5：使用LangGraph

- ◆ 安装LangGraph依赖并在代码中引入依赖
- ◆ 加载文档并进行文档分块
- ◆ 设置嵌入模型和生成模型
- ◆ 创建向量存储（内存存储）
- ◆ 采用新方式定义RAG提示词
- ◆ 自定义应用状态类、检索步骤、生成步骤
- ◆ 构建和编译应用
- ◆ 运行查询输出结果
- ◆ 绘制流程图（可选）

实践6-1：不使用框架(faiss+BGE)

- ◆ 安装faiss依赖并在代码中引入依赖
- ◆ 准备文档数据
- ◆ 设置嵌入模型并嵌入文档
- ◆ 创建向量存储
- ◆ 执行相似度检索
- ◆ 构建提示词
- ◆ 使用Ollama生成答案

实践6-2：不使用框架(zvec+MiniLM)

- ◆ 安装zvec依赖并在代码中引入依赖
- ◆ 准备all-MiniLM-L6-v2嵌入模型并设置嵌入模型
- ◆ 准备文档数据并使用嵌入模型嵌入文档
- ◆ 使用zvec创建Schema创建向量存储
 - 创建结构：Schema
 - 创建数据集合：Collection
 - 将文档的向量数据存入集合
- ◆ 将问题转换为向量后在zvec集合中查询结果
- ◆ 将查询结果转为对应doc输出
- 73 ◆ 构建提示词并使用Ollama生成答案



谢 谢